PIPELINE MINT

DevOps & Cloud [SWE] — Class 17 Notes

LECTURE TITLE	CI/CD & GitHub Actions — the **CI** half: workflows, jobs, steps, runners, secrets, push triggers and artifacts
DATE/TIME	24 September 2026, 2:00 PM
DURATION	120 minutes (scheduled). Her goal was *"we will complete it by 3:30"*. Your own commits show the last push at **4:19 PM**.
INSTRUCTOR	Nensi Ravaliya
BIG IDEA	Stop typing the same build-and-test commands by hand. Write them **once** in a YAML file under `.github/workflows/`, and GitHub runs them for you on a fresh machine every time you `git push`.
SOURCES	the class transcript · her repo folder for this session, `Nency-Ravaliya/devops-heros` → `session-16-github-actions` (`03-github-actions`, `05-jobs-and-steps`, `07-secrets`, `08-artifacts`, `demo/`) · your in-class pipeline repo **`hey-cicd`** (its full git history, 15:15 → 16:19, is now in `Class-18-Complete CI-CD & DevSecOps/hey-cicd/`) · your homework folder `demo/` in this class.

WHAT'S INSIDE

0 ⚡ The 60-Second Version
1 📅 Logistics & Warm-Up
2 🔁 Course Recap Quiz — "So Far, What Did We Learn?"
3 🔄 CI vs CD — What Goes Where
4 🐍 The Calculator App — Doing It All By Hand First
5 🤖 Why We Need a Pipeline — The Argument in Four Steps
6 🧰 The CI/CD Tool Landscape
7 🚀 Your First Workflow — `Hello GitHub Actions`
8 ▶️ Running It & Reading the Logs
9 🔀 Multiple Jobs — Parallel by Default
10 🔐 Secrets — Passwords That Even You Can't Read Back
11 ⚡ Automatic Triggers — Run on Every Push
12 📦 Artifacts — Getting Files Out of a Dying Machine
13 🧪 The Homework Pipeline — `demo/` and `needs`
14 📋 Cheat-Sheet
15 📝 Homework
16 🗒️ Notes on the Transcript Itself

NOTE

The title is right this time. The header says *CI/CD & GitHub Actions*, and that is what was taught. After three wrong titles in a row (Classes 12–14), this one matches. One caveat: only **CI** was done hands-on. **CD** (actually deploying somewhere) was explained in theory only. Your empty `CD/` folder is waiting for **Class 18 — Complete CI/CD & DevSecOps**.

NOTE

Her session number vs your folder number. In her repo this is `session-16-github-actions`, and she kept saying "session 16" out loud. The artifact is even named `session16-build`. Your folder is `Class-17`. Ignore the mismatch and go by content.

NOTE

Where your in-class work lives. The pipeline you built live (`hey-cicd/.github/workflows/cicd.yaml`) lives in this folder (`hey-cicd/`). Its git history is a timestamped record of this class: `CICD added` at 15:24 → `Parallel job` at 15:38 → `Secret` at 15:49 → `Auto` at 16:00 → `Artifact` at 16:19. §8–§12 review that file commit by commit.

---

0 ⚡ The 60-Second Version

1. **CI = Continuous Integration.** Every push gets **built and tested** automatically. "*Before deployment, anything will be there — you put it inside CI pipeline.*"
2. **CD = Continuous Delivery / Deployment.** CI's **output** is CD's **input**. CD puts the app on a server: EC2, EKS, GKE, AKS, OpenShift, or your own cluster. **Delivery** has a **manual approval** gate before production. **Deployment** has none.
3. **Why bother?** Running `pip install`, then `pytest`, then `python3 app/calculator.py` after every change is repetitive. A `build.sh` script helps one person, but you can't email it to 70,000 developers. The automation has to live **inside version control** and run on `git push`.
4. **GitHub's one hard rule:** workflow files **must** be in `.github/workflows/`. Any file name works, but outside that folder nothing runs.
5. **The skeleton:** `name` → `on` (trigger) → `jobs` → each job has `runs-on` (the **runner**) and `steps`. A step either `run`s a shell command or `uses` a ready-made action such as `actions/checkout@v6`.
6. **Jobs run in parallel by default.** Add `needs: <job>` to make one job wait for another. That draws the connecting line in the Actions graph.
7. **Secrets** live in *Settings → Secrets and variables → Actions*. Read them with `${{ secrets.NAME }}`. Once saved, **nobody can view the value again**, not even you.
8. **Artifacts:** the runner is thrown away when the job ends. `actions/upload-artifact@v4` saves files (build output, reports) as a downloadable **zip** on the run page.
9. **Triggers:** `workflow_dispatch` adds a manual **Run workflow** button. `on: push: branches: [main, 'feature/*']` makes the pipeline run on every push.

---

1 🗒️ Logistics & Warm-Up

The broken projector. One side of the room couldn't see the screen. In Hindi:

> SAID IN CLASS
>
> *"This projector is off, so everyone sitting on that side, come to this side, otherwise I won't take your attendance. What can I do? That's the only option I have — attendance."*

The mic. Several minutes went into swapping microphones. "*This mic is uncomfortable. Honestly, but it's okay. This mic is better than that.*" The transcript turns into noise for a stretch while she sorted it out.

The time plan. "*How many hours are you? 2:30. We will complete it by 3:30.*" Later: "*2:50 to 3:30 — do you want to finish class?*" Your commits run until **16:19**, so the hands-on part went well past 3:30.

Typing one-handed. "*See, I'm typing by one hand.*" Most of the demo was typed with one hand.

Who knows Python? She asked who was in the Python stream before picking the demo. The app is a tiny Python calculator, so very little Python knowledge was needed.

An interruption. Near the artifacts section someone came in about a meeting ("*Shithith sir*", "*director*", "*Dean*"; names unverified). She explained she had a meeting with the director and the Dean, then went back to teaching.

The close. She compared the two sections. In **Section B** (the morning batch) she'd had to repeat things two or three times. Then:

> SAID IN CLASS
>
> *"Brilliant batches. Both the batches. No doubt. Most of the people I can see, some people already know the GitHub CI/CD concepts… You're already ahead of the crowd."*

> CLASS MOMENT
>
> **Class moment — the repo name.** She asked the room what to call the demo repo. Someone said **"hey-cicd"** and it stuck: "*Oh my god, I have fun name — hey-cicd.*" She then pictured giving a serious presentation and opening "*my latest project, hey-cicd*" and laughed. Your repo has the same name.

---

2 🔁 Course Recap Quiz — "So Far, What Did We Learn?"

The class opened with a team quiz (Teams A, B, C, D) recapping the course so far. She wanted the topic list, then everything about Docker, then everything about Kubernetes, then the list of **Kubernetes objects**.

> NOTE
>
> **This is a deliberate repeat.** Almost every question below was taught in an earlier class. Following the repo rule, the answers are written out in full here rather than pointing back. This PDF should work on its own.

2.1 The quiz, as it happened

#	Question	Who	Answer given	Verdict
1	What did we learn first in DevOps?	Room	**Linux**	✅
2	After Linux?	Room	**Shell scripting**	✅
3	Then?	Room	**GitHub**	✅
4	After GitHub?	Room	Stuck on "*GitHub… GitHub did it*"	❌ She answered herself: "*Then just forget the networking.*" It was **Networking**
5	Then?	Room	**Docker**	✅
6	How many Docker objects? Name them	Room	**Network, Volume, Container, Image**	✅
7	What did we learn in Kubernetes?	Room	Silence. "*Either you give me an answer or…*" Then the seating break (§1)	❌ at first
8	(retry) Kubernetes concepts, one by one	Room	**Control plane**, **worker node**, **master node**, **etcd**, **kubectl**, **Minikube**, **port forwarding**, **scheduler/scheduling**, **API server**, **ReplicaSet**, **DaemonSet** ("*nice*")	✅
9	List the Kubernetes **objects**, the things you put in `kind:`	Room	**Pod** → **Service** → **Deployment**	✅
10	The fourth object?	**Team D**, put on the spot ("*I will count 3*")	Long silence, then **ReplicaSet**	✅ (slowly)
11	More objects	Room	**StatefulSet** ("*nice*"), **DaemonSet**, **ConfigMap**	✅
12	(someone) **Endpoint**?	Room	"*End point is not one of the Kubernetes objects… we have learned that, but it is not one of the objects.*"	❌ in her quiz (see 🔍 below)
13	The rest of the list	She completed it	**Service, ConfigMap, Secret, Ingress, PV, HPA, PVC**	—
14	Other Kubernetes concepts (not objects)	Room	**Endpoint**, **Volume**, **types of services**	✅
15	List the types of Service	Room	**LoadBalancer, ClusterIP, Headless, ExternalName**	⚠️ partial: **NodePort** was missed (Class 13 taught five)
16	How many deployment strategies so far?	Room	**Rolling update, Blue-Green, Canary, Recreate**	✅

CLARIFIED

> **Clarified: Endpoint *is* a Kubernetes API object, just not one you write.** For *her* quiz, she wanted the objects you put in `kind:` yourself (Pod, Deployment, Service…), so give her answer. Technically, though, `kubectl get endpoints` works because `Endpoints` (and the newer `EndpointSlice`) are real API objects. Kubernetes creates and updates them **automatically** for each Service, listing the pod IPs behind it. You almost never write one by hand. That's the honest reason it felt different from the rest of the list.

2.2 The answers, written out in full

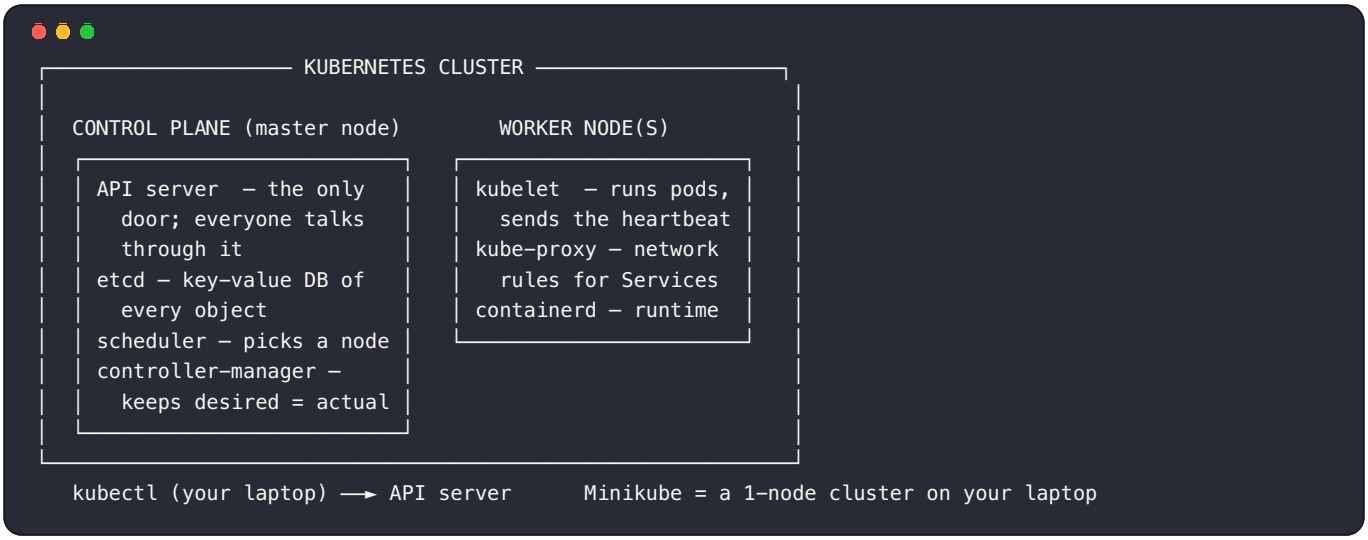
The course so far, in order:

```
Linux → Shell scripting → Git & GitHub → Networking → Docker → Kubernetes → Helm → (today) CI/CD
```

The four Docker objects:

Object	One line	Analogy
Image	Read-only template: code + runtime + dependencies, in layers	The recipe
Container	A running instance of an image	The dish cooked from the recipe
Network	How containers find and talk to each other (bridge, host, none, custom)	The phone line between kitchens
Volume	Storage that outlives the container	The fridge: food survives after the cook leaves

The Kubernetes architecture (the concepts list from Q8):



Concept	What it is
kubectl	The CLI on your laptop. Every command goes to the API server
Minikube	A local, single-node practice cluster
Port forwarding	`kubectl port-forward` tunnels a local port to a pod/service, for quick testing
Endpoint	The live list of pod IPs behind a Service
Volume	Storage mounted into a pod

The Kubernetes objects (Q9–Q13):

Object	One-line job
Pod	Smallest unit: one or more containers sharing a network
ReplicaSet	Keeps *N* copies of a pod alive
Deployment	Manages ReplicaSets, adds rolling updates and rollback
StatefulSet	Like a Deployment, but with stable names (`mysql-0`) and per-pod storage, for databases
DaemonSet	Exactly one pod on **every** node (log agents, monitoring)
Service	A stable address in front of changing pods
ConfigMap	Non-secret settings (`LOG_LEVEL` , `PORT`)
Secret	Sensitive settings (passwords, tokens), base64-encoded
Ingress	One entry point that routes by path/host to many Services
PV / PVC	PersistentVolume = the actual disk; PersistentVolumeClaim = a pod's request for one
HPA	Horizontal Pod Autoscaler: adds/removes pods based on CPU/memory

Service types (Q15). The full five:

Type	Reachable from
ClusterIP (default)	Inside the cluster only
NodePort ← *missed in the quiz*	`<node-IP>:<30000-32767>` from outside
LoadBalancer	A cloud load balancer's public IP (`<pending>` on Minikube)
ExternalName	A DNS alias to something outside the cluster
Headless (`clusterIP: None`)	No single virtual IP; DNS returns pod IPs directly (used with StatefulSets)

Deployment strategies (Q16):

Strategy	How it rolls out	Downtime?
Rolling update (default)	Replace pods a few at a time (`maxSurge` / `maxUnavailable`)	No
Recreate	Kill all old pods, then start new	**Yes**
Blue-Green	Run old (blue) and new (green) side by side, flip traffic at once	No, but double the resources
Canary	Send a small % of traffic to the new version first, then grow	No

3 🔄 CI vs CD — What Goes Where

PICTURE THIS

Picture a restaurant.

CI is the kitchen: every new dish is cooked (**build**) and tasted (**test**) the moment the recipe changes. A bad dish never leaves the kitchen. **CD** is the waiter carrying the plate to the table (**deploy**). **Continuous *Delivery*** means the waiter waits for the manager's nod before serving. **Continuous *Deployment*** means it goes straight out.

3.1 The two letters

	CI — Continuous Integration	CD — Continuous Delivery / Deployment
Job	Build + test every change	Release the tested output somewhere real
Contains	Everything **before** deployment	The deployment itself
Input	Your source code	**CI's output**
Example (Node.js)	`npm install` → `npm run build` → `npm run test`	Copy the build to a server / cluster
Example (today's Python app)	`pip install -r requirements.txt` → `pytest`	(next class)

SAID IN CLASS

"In a CI stage, we only add our build and test things… Before deployment, anything will be there, you put it inside CI pipeline. And in a CD pipeline… whatever you want to deploy — from CI pipeline you will get the output, that is the input for your CD pipeline."

Where CD can deploy to (her list):

- an **EC2** instance · your **localhost** · an **Azure** server
- **EKS** (AWS) · **GKE** (Google Kubernetes Engine) · **AKS** (Azure)
- an **OpenShift** cluster
- **your own Kubernetes cluster** on your own servers. "*Because it's open source, right? You can use any of your server and you can build your own Kubernetes cluster.*"

3.2 Delivery vs Deployment

She asked the room ("*I told you, in the first-ever session*") and gave the key: **one of the two has a manual approval step.**

CLARIFIED

Clarified: which one has the approval. In class it came out as "*in continuous deployment or in a continuous delivery, one of these… we need to add a manual approval.*" To pin it down:

	Continuous Delivery	Continuous Deployment
After tests pass	App is *ready* to release	App *is* released
Production push	**Manual approval** (a human clicks "approve")	**Automatic**, no human
Typical in	Banks, regulated industries	SaaS shipping many times a day

Her own repo's `01-ci-vs-cd/README.md` says the same: *Delivery* → "automatically **deployable**"; *Deployment* → "automatically **deployed** to production".

3.3 Why do we need CI/CD at all?

A student answered: **"to automate the testing and build."** She accepted it: "*That is one of the answers.*" §5 builds the full argument from the calculator demo.

INTERVIEW ALERT

Interview alert. "*What's the difference between CI, Continuous Delivery and Continuous Deployment?*" CI = build + test on every commit. Delivery = always releasable, with a human approval before prod. Deployment = every green build goes to prod automatically.

---

4 🐍 The Calculator App — Doing It All By Hand First

Before any pipeline, she made everyone run the app manually so the pain of repetition is felt.

CLASS MOMENT

Class moment. "*How many of you have built a calculator application in Python? … I thought we would take a lot of tough code. So I took a calculator code — very tough. Where we will do 2 plus 3 equals 5.*" Then: "*No one will add any modulo operation. No one will divide by 0. If you don't make my code work, I won't make your code work.*"

4.1 The project

```
demo/
├── app/
│   ├── __init__.py
│   └── calculator.py        ← 4 operations: add, subtract, multiply, divide
├── tests/
│   └── test_calculator.py   ← 5 pytest tests
├── requirements.txt         ← just one line: pytest
├── build.sh                 ← packages the app into build/
└── .github/workflows/ci.yml ← the homework pipeline (§13)
```

She cleaned it up live: "*forget about this first folder,* `__pycache__` *— I removed it.*"

`app/calculator.py`: four functions plus an input loop that keeps asking until you type `q` or `quit`:

```
def add(a, b):      return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b
def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

(The full file adds a `while True:` loop that parses input like `10 + 5` with a regex and prints `Result: 15.0`. The `.0` is there because inputs are converted with `float()`. She called it "*I mean decimal, my output will get 5*".)

`requirements.txt` contains just `pytest`. "*Why are you using pytest? You will know if you are a Python developer.*" It's the test runner.

4.2 Running it

```
# 1. create + activate a virtual environment (she pasted these two in chat)
python3 -m venv .venv
source .venv/bin/activate

# 2. install dependencies
pip install -r requirements.txt

# 3. run the app — from the project root, the file is inside app/
python3 app/calculator.py
```

Live session:

```
Calculator Application
----------------------
Available operations: +, -, *, /
Type 'q' or 'quit' to exit.

Enter calculation (e.g., 10 + 5): 2 + 1
Result: 3.0
Enter calculation (e.g., 10 + 5): 2 - 1
Result: 1.0
Enter calculation (e.g., 10 + 5): 2 * 3
Result: 6.0
Enter calculation (e.g., 10 + 5): q
Goodbye!
```

PICTURE THIS

What a virtual environment is. A private `site-packages` folder just for this project. `pip install` puts `pytest` in `.venv/`, not in your system Python, so two projects can need different versions without a fight. `source .venv/bin/activate` makes `python` / `pip` point at that folder for this terminal session.

4.3 Running the tests

```
pytest        # short output — a row of dots and "5 passed" in green
pytest -v     # verbose — one line per test
```

```
tests/test_calculator.py::test_add PASSED
tests/test_calculator.py::test_subtract PASSED
tests/test_calculator.py::test_multiply PASSED
tests/test_calculator.py::test_divide PASSED
tests/test_calculator.py::test_divide_by_zero PASSED
============================== 5 passed ==============================
```

The five tests in `tests/test_calculator.py`:

Test	Asserts
`test_add`	`add(10, 5) == 15`
`test_subtract`	`subtract(10, 5) == 5`
`test_multiply`	`multiply(10, 5) == 50`
`test_divide`	`divide(10, 5) == 2`
`test_divide_by_zero`	`divide(10, 0)` **raises** `ValueError` (`pytest.raises`)

WATCH OUT

If your tests fail here, you copied the wrong files. "*If your test is failing, that means you copied a wrong file.*" The test file imports `from app.calculator import …` and adds the project root to `sys.path`, so the folder layout has to match exactly.

DO THIS YOURSELF

Lab 1 — run it all by hand.

1. Clone her repo and copy the demo folder somewhere fresh:

```
git clone --depth 1 https://github.com/Nency-Ravaliya/devops-heros.git
cp -r "devops-heros/session-16-github-actions/session-16-github-actions/demo" ~/calc-demo
cd ~/calc-demo
```

1. `python3 -m venv .venv && source .venv/bin/activate`
2. `pip install -r requirements.txt`
3. `python3 app/calculator.py`. Try `2 + 1`, `7 / 2`, `5 / 0` (you should see `Error: Cannot divide by zero`), then `q`.
4. `pytest` → expect **5 passed**. Then `pytest -v`.
5. `./build.sh` (run `chmod +x build.sh` first if needed) → `cat build/build-info.txt`.
6. Change one character in `calculator.py` and do steps 3–6 **again**. Notice how annoying that is. That annoyance is §5.

---

5 🤖 Why We Need a Pipeline — The Argument in Four Steps

5.1 Step 1: the repetitive task

SAID IN CLASS

"Every time if I'm doing some changes inside my code, then I need to again run these two commands… so this is one of the repetitive tasks. How can we neglect it, or how can we convert it in a script?"

Every edit means: `pip install -r requirements.txt` → `pytest` → `python3 app/calculator.py`.

5.2 Step 2: the obvious fix, a script (`build.sh`)

"One of the solutions that we found right now is we can create one automation script." Here is the `build.sh` she shipped (identical in your `demo/`):

```
#!/bin/bash
set -e                                  # stop at the first failing command
echo "Starting build..."
rm -rf build                            # start clean every time
mkdir -p build
cp app/calculator.py build/             # "package" the app
echo "Application copied to build directory."
cat > build/build-info.txt <<EOF
Application: Session 16 Calculator
Build Status: SUCCESS
Build Date: $(date)
EOF
echo "Build completed successfully."
```

Her walkthrough: it copies `calculator.py` into `build/`, then writes `build-info.txt` with three lines: the app name, the status, and **the date**. "*Every time it will print a new date and time… so every time I will get the exact date and time when I did changes inside my code base.*"

Your own run proves it. `demo/build/build-info.txt` says:

```
Application: Session 16 Calculator
Build Status: SUCCESS
Build Date: Thu Sep 24 15:00:39 IST 2026
```

That's 3:00 PM, an hour into class.

CLARIFIED

Clarified: what `build.sh` actually contains. She described it as "*whatever I'm running, three commands, I'm just putting inside my build.sh*", but the real file **doesn't** run `pip install`, `pytest` or the app. It only **packages** (copy + write build info). In the pipeline those other commands become **separate steps** (§13): install → test → then `build.sh`. The idea is the same (script the repetitive part). The file is just narrower than the description.

LOCK THIS IN

Nice detail in that script. The heredoc ends `<<EOF` with **no quotes**, so `$(date)` is **expanded** when the script runs. If you wrote `<<'EOF'` you'd get the literal text `$(date)` in the file. That's the same quoting idea as the `$(date)` bug in Class 3 §11, here used correctly.

5.3 Step 3: the script doesn't scale

SAID IN CLASS

"*What if I tell you that now the 10 developers are working in that project, and now you are going to share that build.sh file to the 10 developers. Is it a good approach? No.*"

"*Now I have a big infra where I have a Node.js application, Java application, Maven, Gradle, Python, a lot of microservices… maybe 50, 70 thousand developers are working. So do we need to create a build.sh file at that time? Is it the right solution? No.*"

5.4 Step 4: put the automation inside version control

SAID IN CLASS

"We need some type of automation inside of a version control system, where we are using that automation, we can track those changes, and anyone can use it directly without even much conflict. So this is the thing that we have — a CI pipeline."

WATCH IT HAPPEN

Watch it happen — what changes once CI exists.

```
BEFORE                                  AFTER
──────                                  ─────
edit code                               edit code
pip install -r requirements.txt         git add . && git commit -m "…"
pytest                                  git push
python3 app/calculator.py                  │
./build.sh                                 ▼
(repeat on every laptop)                GitHub sees the push
                                           ▼
                                        spins up a fresh Ubuntu runner
                                           ▼
                                        pip install → pytest → build.sh
                                           ▼
                                        ✅ green tick  /  ❌ red cross
                                        (same result for every developer)
```

SAID IN CLASS

"Once I will do git push, it will run, it will do the pip install -r requirements.txt, it will do the test cases and it will execute my Python application… I don't need to run even build.sh file. Whenever I do git push, it will automatically do it. This is the use case of CI/CD pipeline."

---

6 🧰 The CI/CD Tool Landscape

She asked the room to name tools, then filled in the rest:

Tool	From	Cost / notes (as she said them)
GitHub Actions	GitHub	Free tier; enterprises pay. **Today's tool**
Jenkins	Open source	"*Completely free tool, open source code*". The "*traditional tool*" for CI/CD
Azure DevOps	Microsoft	"*In Azure we have a complete DevOps portal*"
CircleCI	CircleCI	Named in the list
AWS CodeBuild + CodeDeploy	AWS	"*These are the two services that we are using in AWS*"

GOOD TO KNOW

Filled in (not said in class): AWS has a third piece, **CodePipeline**, which chains CodeBuild and CodeDeploy into one pipeline. **GitLab CI** is the other big name you'll meet in job posts.

6.1 Jenkins in a container (her tip for the curious)

SAID IN CLASS

"You don't need to install Jenkins. If you want to run the same pipeline over the Jenkins, you just need to pull a Docker image for Jenkins… Then after pulling the image, there is a port number for Jenkins. You just check the port number in your browser and you will see Jenkins is running. You need to reset a password… And then you can create thousands or millions of projects over that one container. Can you believe? … You need to increase the container size, the amount of RAM."

GOOD TO KNOW

Filled in, so you can actually do it:

```
docker pull jenkins/jenkins:lts          # the maintained image ("jenkins" alone is deprecated)
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
# open http://localhost:8080 and paste that password to unlock it
```

Port **8080** is the web UI, **50000** is for build agents. The `-v jenkins_home:…` named volume is Class 8 knowledge: without it, your jobs vanish when the container is removed.

6.2 Every platform has its own file rule

Platform	Where the pipeline file must live
GitHub Actions	`.github/workflows/<anything>.yml`. *"If you are not putting your CI/CD pipeline inside `.github/workflows`, then your pipeline will not work, guys. This is the rule."*
Azure DevOps	*"You must name the pipeline as `azure-pipelines.yml`"*
Jenkins	*"You don't have any naming convention… because it's its own platform"*

CLARIFIED

Clarified, lightly. `azure-pipelines.yml` is Azure DevOps' **default** name (it auto-detects it), but you can point a pipeline at any YAML path. Jenkins *convention* is a file called `Jenkinsfile` at the repo root, though you can configure another path. **GitHub's rule is the strict one.** `.github/workflows/` is not optional. That's the one to remember for her course.

SAID IN CLASS

On difficulty: "*GitHub Actions is one of the tough platforms that I found for myself in past 5 years… Azure DevOps — whenever you are writing a file in Azure DevOps YAML it will be very easy. Jenkins files are all easy… This is one of the tough ones, but yeah, we'll do it.*"

---

7 🚀 Your First Workflow — `Hello GitHub Actions`

7.1 Setting up the repo

1. On GitHub, create a new repo: `hey-cicd`.
2. Clone it: `git clone <repo-url>` (she cloned onto her Desktop).
3. Open the folder in your IDE (she used **Antigravity**; VS Code works too).
4. Create the magic folder and a workflow file. "*Any name you can put*":

```
mkdir -p .github/workflows
touch .github/workflows/hey-cicd.yaml       # you named yours cicd.yaml — also fine
```

GOOD TO KNOW

Two ways to get a pipeline. Write your own (today), **or** go to the repo's **Actions** tab and pick one of GitHub's free **starter templates** (Java, Node, Python, Docker, Azure…). "*GitHub Actions has a lot of free predefined templates… and you can use it for free.*"

7.2 The file

Copied from her repo's `03-github-actions` folder:

```yaml
name: Hello GitHub Actions
on:
  workflow_dispatch:
jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - name: Print message
        run: echo "Hello from GitHub Actions!"
      - name: Show date
        run: date
      - name: Show operating system
        run: uname -a
```

SAID IN CLASS

"There is one file. If you learn this file, each keyword, then it will be very easy to create any CI/CD pipeline. Not even in just this platform."

7.3 Keyword by keyword

```
name ──────────────► what shows up in the Actions tab
on ────────────────► WHEN it runs (the trigger)
jobs ──────────────► the big units of work
  hello ───────────► a job's ID (you pick it)
    runs-on ───────► WHERE it runs (the runner)
    steps ─────────► WHAT it runs, top to bottom
      - name ──────► label shown in the logs
        run ───────► the shell command
```

Keyword	Her explanation	Notes
`name`	*"Your pipeline name."*	Shows in the Actions sidebar
`on: workflow_dispatch`	*"It will take my YAML file from workflow folder"*	See 🔍 below. It's the **manual trigger**
`jobs`	*"Either it will be build job, test job, any security jobs… whatever stages you are running inside your pipeline, we call it as a job."*	Node example: a **build** job (`npm install` , `npm run build`) and a **test** job (`npm run test`)
`hello`	The job's name. *"Maybe our job name is build. Maybe test."*	This is the name you click on in the UI
`runs-on: ubuntu-latest`	*"My job, my pipeline will run on Ubuntu latest… we call it as a runner."*	See 🔍 below. It's a VM
`steps`	*"The commands that we are executing."*	Run **in order**, top to bottom
`- name:` / `run:`	Label + command	`echo` , `date` and `uname -a` work *"because it's running on Ubuntu"*

CLARIFIED

Clarified: `workflow_dispatch` **means "give me a Run button".** In class it was described as *"it will take my YAML file from workflow folder"*. That isn't what it does. **Every** file in `.github/workflows/` is picked up automatically. `workflow_dispatch` is a **trigger**: it adds a **"Run workflow"** button in the Actions tab so you can start the pipeline **manually**. She confirms this herself a few minutes later: *"Which one we did right now? Manually."* Other triggers you'll meet: `push` , `pull_request` , `schedule` (cron), `release` .

CLARIFIED

Clarified: `ubuntu-latest` is a virtual machine, not a Docker image. She said "*Ubuntu latest is the Docker image. In this Docker image, my pipeline will run.*" The mental model (a throwaway, pre-built environment) is right. But GitHub-hosted runners are **fresh virtual machines**, one per job, deleted afterwards. That's why Docker itself is installed and usable *inside* a GitHub runner. For her assessment, "runner = the machine the job runs on; `ubuntu-latest` = GitHub's hosted Ubuntu runner" is the safe answer.

7.4 Runners: hosted vs your own

SAID IN CLASS

"Custom runner is one of the advanced topics that no one is aware [of]. If person is aware, then they don't know how to implement it actually."

Her story: years ago she **made an EC2 instance into an agent** and connected it to her pipeline, "*so whatever the pipeline I'm running, that pipeline will run inside my server*" instead of on GitHub's machines.

	GitHub-hosted runner	Self-hosted (custom) runner
`runs-on:`	`ubuntu-latest`, `windows-latest`, `macos-latest`	`self-hosted` (plus your labels)
Who owns the machine	GitHub	You (EC2, on-prem box, your laptop)
Fresh every job?	**Yes**	No. State can linger between runs
Cost	Free minutes, then pay	You pay for the server; no GitHub minute limit

Why bother with your own? The free plan only gives you a limited number of minutes:

SAID IN CLASS

"If you have run one pipeline which has taken 15 minutes… you run the pipeline 100 times, it will count the time. It will exceed … in a month. Then you can't run the CI/CD pipeline. It is predefined hours."

GOOD TO KNOW

Filled in, with the exact numbers. 15 min × 100 runs = **1,500 minutes = 25 hours**. On GitHub's **Free** plan, **private** repos get **2,000 runner-minutes per month**. Standard hosted runners on **public** repos are free with no minute cap. (The audio said "more than 13 hours"; the arithmetic above is the precise version.)

---

8 ▶️ Running It & Reading the Logs

8.1 Push, then press the button

```
git add .
git commit -m "CICD added"
git push
```

Then on GitHub: **Actions** tab → click **Hello GitHub Actions** (the `name:`) → **Run workflow** → branch **main** → green **Run workflow** button.

Dot	Means
🟡 yellow	Running / queued
✅ green tick	Success
❌ red cross	Failed

8.2 What you see inside the run

Click the run → click the job `hello` ("*why do we see hello? Because we gave the job name as hello*"). You'll see **five** steps, although you only wrote three:

```
✅ Set up job                ← added automatically
✅ Print message             ← yours
✅ Show date                 ← yours
✅ Show operating system     ← yours
✅ Complete job              ← added automatically ("post job")
```

> **SAID IN CLASS**
>
> *"Setup job means it will initialize your CI/CD pipeline… it will set up your operating system, Ubuntu latest. And last job is complete job. In a complete job it will clean up all the activities that you did. We call it as a post job."*

Expand each step to see **the command (input)** and **its output**:

Step	Command	Output
Print message	`echo "Hello from GitHub Actions!"`	`Hello from GitHub Actions!`
Show date	`date`	the date/time **in UTC** (runners use UTC, not IST)
Show operating system	`uname -a`	`Linux …`

Expanding the command header shows *where* it ran: `shell: /usr/bin/bash -e {0}`. "*So you can run [any] command.*"

> **WHY IT WORKS**
>
> **Why `bash -e`.** The `-e` flag means "exit on the first error", the same as `set -e` in `build.sh`. If any command in a `run:` block fails, the step fails, the job fails, and you get the red cross. That non-zero exit code is how the whole CI system knows something broke.

> **WATCH IT HAPPEN**
>
> **Watch it happen — one manual run, frame by frame.**

```
Frame 1  You click "Run workflow" on main
Frame 2  GitHub queues the job "hello"                          🟡
Frame 3  A fresh ubuntu-latest VM boots      → "Set up job"
Frame 4  bash -e runs: echo "Hello…"          → Print message ✅
Frame 5  bash -e runs: date                   → Show date ✅  (UTC)
Frame 6  bash -e runs: uname -a               → Show OS ✅    (Linux)
Frame 7  Cleanup                              → "Complete job"
Frame 8  VM is destroyed. Run marked                            ✅
```

8.3 The "Show myself" exercise

TRY IT YOURSELF

Her task: "*Add one more stage here. You will name it as Show myself… then inside that run command, do the echo, hello and your name.*" Then `git add`, `git commit`, `git push`, run it, and show her on your laptop.

```yaml
      - name: Show myself
        run: echo "Hello, <your name>"
```

NOTE

Review of your file (`hey-cicd` , commits 15:29 → 15:31). Your first try was:

```yaml
      - name: Show myself
        echo: Anishka Nase        # ❌ 15:29
```

`echo:` isn't a step keyword. A step only understands keys like `name`, `run`, `uses`, `with`, `env`, `if`. GitHub rejects the file as an invalid workflow. Two minutes later you fixed it to `run: echo "Anishka"` ✅. Remember the rule: **the command always goes after `run:`**, and `echo` is part of the command.

DO THIS YOURSELF

Lab 2 — your first pipeline, end to end.

1. Create a new **public** GitHub repo (public means unlimited free minutes). Clone it.
2. `mkdir -p .github/workflows` and paste the §7.2 YAML into `.github/workflows/hello.yml`.
3. Add a fourth step, `Show myself`, that echoes your name.
4. `git add . && git commit -m "first pipeline" && git push`.
5. **Actions** → **Hello GitHub Actions** → **Run workflow** → main.
6. Open the run, click `hello`, and count the steps (5). Expand `Show date` and convert the UTC time to IST (+5:30).
7. **Break it on purpose:** add a step `run: exit 1`. Push, run, and watch the red ❌. Steps after it are **skipped**. Remove it again.

9 🔀 Multiple Jobs — Parallel by Default

9.1 What she did

"*This is our first job, hello job. So I will copy my job, I will name it as build.*" Push, run, and now the run page shows **two** jobs side by side:

```yaml
jobs:
  hello:
    runs-on: ubuntu-latest
    steps: [ ...same 3–4 steps... ]
  build:                                # ← copy-paste of hello, renamed
    runs-on: ubuntu-latest
    steps: [ ...same steps... ]
```

```
Run graph:

   ┌─────────┐
   │  hello  │ ✅
   └─────────┘            ← no line between them =
                            they started at the same time
   ┌─────────┐
   │  build  │ ✅
   └─────────┘
```

She also pointed at the summary: **"Artifacts: none"**, "*Can you see artifacts are null?*", as the setup for §12.

> **WHY IT WORKS**
>
> **Why jobs don't share anything.** Each job gets its **own fresh runner VM**. `hello` and `build` run on two different machines at the same time. A file created in `hello` does **not** exist in `build`. That's why jobs are independent by default, and why artifacts (§12) exist.

9.2 Making one wait: `needs`

She didn't teach `needs` live (it shows up in the homework, §13), but **you used it early**. Your 15:40 commit "Parallel job" added a third job:

```yaml
  build2:
    runs-on: ubuntu-latest
    needs: build          # wait for "build" to succeed first
```

```
With needs:   build ──► build2        (a connecting line in the graph)
Without:      build     build2        (side by side)
```

NOTE

Review of your file: the rename that broke `needs` (16:18 → 16:19). At 16:18 you renamed the job `build:` → `build1:` but left `build2` saying `needs: build`. A `needs:` that points at a job ID that no longer exists makes the **whole workflow invalid**. GitHub refuses to run it. One minute later you commented the line out (`# needs: build`), which made the file valid again but quietly turned the chain back into parallel jobs. **The real fix is `needs: build1`.** Lesson: job IDs are references. Rename one and every `needs:` pointing at it must change too.

DO THIS YOURSELF

Lab 3 — parallel, then chained.

1. Copy your `hello` job, rename it `build`, push, and run. Confirm two jobs with no line between them.
2. Add a third job `deploy` with `needs: build` and one step `run: echo "deploying…"`. Push and run. Now there's a line `build → deploy`.
3. Add `run: exit 1` as the last step of `build`. Push and run. `deploy` shows as **skipped**.
4. Rename `build` → `build1` **without** updating `needs`. Push. Read the "Invalid workflow file" error, then fix it properly.

---

10 🔐 Secrets — Passwords That Even You Can't Read Back

"Now two concepts only left for today's session: one is artifact and one is secret."

10.1 The secret-demo job

From her repo's `07-secrets` folder, pasted as a **new job** in the same file:

```yaml
  secrets-demo:
    runs-on: ubuntu-latest
    steps:
      - name: Check secret
        env:
          DEMO_SECRET: ${{ secrets.DEMO_SECRET }}
        run: |
          if [ -n "$DEMO_SECRET" ]; then
            echo "Secret is available."
          else
            echo "Secret is not configured."
            exit 1
          fi
```

Line	What it means
`env:`	"*Another predefined keyword… we are using to get the environment variable.*" Sets env vars for this step
`DEMO_SECRET:`	The **environment variable name** inside the runner
`${{ secrets.DEMO_SECRET }}`	The **value**, pulled from the repo's secret store. `${{ … }}` is GitHub's expression syntax
`if [ -n "$DEMO_SECRET" ]`	"Is this variable **non-empty**?"
`exit 1`	Fail the step on purpose if the secret is missing

PICTURE THIS

`-n` in one line. `[ -n "$X" ]` is true when `$X` has **n**on-zero length. Its opposite is `[ -z "$X" ]` (**z**ero length). A secret that was never created comes through as an empty string, so `-n` is false, you land in `else`, and `exit 1` turns the step red.

INTERVIEW ALERT

The name that matters is the one after `secrets.` She stressed this. `DEMO_SECRET:` on the left is just the env-var name in the runner. `secrets.DEMO_SECRET` on the right is the **actual secret name you must create** in Settings. Here both are `DEMO_SECRET`, but they don't have to match:

```
env:
  DB_PASS: ${{ secrets.POSTGRES_PASSWORD }}   # create a secret called POSTGRES_PASSWORD
```

10.2 Fail first, on purpose

"*Right now we did not create a secret. That means if I'm running this pipeline, this should be failed, right?*" Push → Run workflow → three jobs: `build`, `hello`, `secrets-demo` ❌. "*It failed. Is it failing in your repo too?*"

10.3 Creating the secret

WATCH IT HAPPEN

Watch it happen — adding a repository secret.

```
Frame 1  Repo → ⚙ Settings
Frame 2  Left sidebar, scroll down → "Secrets and variables"
Frame 3  Choose  Actions   (not Agents / Codespaces / Dependabot)
         "because it's GitHub Actions"
Frame 4  "New repository secret"
Frame 5  Name:  DEMO_SECRET          ← exactly what follows `secrets.`
         Value: Nancy_PWD            ← her demo value
Frame 6  "Add secret" → it appears in the list, locked 🔒
Frame 7  Click edit → the value field is EMPTY. You can replace it,
         never read it.
```

SAID IN CLASS

"Once I created my secret and I just hit an add secret, after that no one can see your secret value. Even not you. I can even just update my secret value. I cannot see what I put in my secret last time." And the reason: *"If someone hacked my GitHub repository… they are not able to check my secret value."*

What goes in secrets (her examples): a **Postgres username/password**, a **JWT token**, **TLS/SSL certificates and keys** (`.crt`, `.key`).

GOOD TO KNOW

Also good to know (filled in). GitHub **masks** secret values in logs. If a step accidentally `echo`s one, the log shows `***`. Compare with Kubernetes Secrets (Class 14): those are only **base64-encoded**, so anyone who can read the object can decode them. GitHub secrets are **encrypted** and write-only from the UI.

10.4 Re-run only what failed

Back on the failed run → **Re-run jobs** ▾ → two options:

Option	Does
Re-run all jobs	Everything again
Re-run failed jobs ✅	Only `secrets-demo`. The one she picked

Result: ✅ `Check secret` → `Secret is available.`

NOTE

Review of your file: the indentation bug (15:49 → 15:56). Your first paste looked like this:

```
  secrets-demo:
  runs-on: ubuntu-latest     # ❌ same indent as the job name
  steps:
```

YAML reads that as **three separate jobs** called `secrets-demo`, `runs-on` and `steps`, and none of them is valid. Seven minutes later you indented `runs-on` / `steps` two more spaces ✅. In YAML, **indentation is structure**: the job's settings must sit *inside* the job, one level deeper.

DO THIS YOURSELF

Lab 4 — secrets.

1. Add the `secrets-demo` job. Push and run. It should **fail** with `Secret is not configured.`
2. Settings → Secrets and variables → Actions → New repository secret → `DEMO_SECRET` = any fake value.
3. On the failed run: **Re-run failed jobs** → ✅ `Secret is available.`
4. Add a step `run: echo "$DEMO_SECRET"` (with the same `env:`). Run it and confirm the log shows `***`.
5. Go back to the secret, click edit, and confirm you **cannot** see the value.

11 ⚡ Automatic Triggers — Run on Every Push

"Now do you want to learn how to run your pipeline automatically? You are doing it manually, correct?"

```yaml
name: Hello GitHub Actions
on:
  push:
    branches:
      - main
      - 'feature/*'
jobs:
  ...
```

SAID IN CLASS

"Once you are pushing something inside your main branch, your pipeline will automatically execute… Can you see `feature/` and asterisk sign there? What does it mean? Any feature branch that we have — if you are doing git commit and git push in that branch, this pipeline will automatically run."

Pattern	Matches
`main`	only `main`
`feature/*`	`feature/login`, `feature/cart` … (one level)
`feature/**`	also `feature/ui/navbar` (any depth). *Filled in*

How to see it fired by itself: push, then look at the **repo's home page**. Next to your latest commit message there's a 🟡 dot that turns ✅. *"Can you see this pending mode is there, yellow color dot?"* Click it, or open Actions. Her run was titled with her commit message, **"secret added"**: *"I did not run it manually."*

WATCH OUT

"You can't add two `on` keywords." True: YAML keys must be unique, so a second `on:` line overwrites or breaks the first. She told the class to *remove* `on: workflow_dispatch` and *replace* it with `on: push`.

CLARIFIED

Clarified: you can have both triggers, under one `on:`. You don't have to choose. List several triggers **inside a single `on:`**:

```yaml
on:
  push:
    branches: [main, 'feature/*']
  workflow_dispatch:          # keep the manual button too
```

That's exactly what your `hey-cicd` file did at 15:58, and what her homework `ci.yml` does (with `pull_request` as a third trigger). If you *remove* `workflow_dispatch`, the **Run workflow** button disappears. "Two `on` keys" is wrong; "two triggers under one `on`" is fine.

NOTE

Review of your file: `branch:` vs `branches:`. Your trigger (still in the final file) says:

```
push:
  branch:          # ❌ should be  branches:
    - main
    - feature/*
```

GitHub's workflow syntax only knows `branches` (plus `branches-ignore`, `tags`, `paths`). `branch` isn't a valid key. If your 16:00 "Auto" commit showed an *Invalid workflow file* banner, or didn't run at all, this is why. Change it to `branches:` and also quote `'feature/*'`. YAML treats `*` specially at the *start* of a value, so quoting glob patterns is a good habit.

DO THIS YOURSELF

Lab 5 — automatic runs.

1. Put both `push` (branches `main` and `'feature/*'`) and `workflow_dispatch` under one `on:`.
2. Push to `main`. Watch the 🟡 → ✅ dot appear on the repo home page **without** clicking anything.
3. `git switch -c feature/test && git commit --allow-empty -m "try feature" && git push -u origin feature/test` → it runs again.
4. `git switch -c experiment && git push -u origin experiment` → it **doesn't** run (no matching pattern).
5. Confirm the **Run workflow** button is still there.

---

12 📦 Artifacts — Getting Files Out of a Dying Machine

12.1 Why

SAID IN CLASS

"Let's suppose that CI/CD pipeline is generating some HTML report, maybe some PDF, maybe some code files, maybe a JSON file… and that output you need it inside your CD pipeline, or maybe you need it for your use cases. So how you get those files? For that reason, we need artifacts. … It's kind of a storage where you can directly download your files."

HARD MODE

Her real-world example: the security agent. At work they run a **security agent** in the pipeline that scans the codebase for **vulnerabilities**: deprecated libraries, known **CVE**s. It produces a **PDF / HTML report**: *"in line number this and file number this, this is the error."* As the DevOps engineer, *"your task is to generate those PDFs and then hand over to the development person"*. They fix it and re-run the pipeline. The report reaches them as an **artifact**. (This is exactly where Class 18, *DevSecOps*, picks up.)

PICTURE THIS

Picture this. The runner is a hotel room that gets demolished the moment you check out. Anything you want to keep, you have to put in the hotel's **left-luggage locker** (the artifact store) before you leave. Afterwards you can collect it as a zip from the run page.

12.2 The new keyword: `uses`

SAID IN CLASS

"Whatever you see — `uses` — this is something new that you don't know, right? You know only the `name` and `run` command… `uses` means already predefined library, predefined actions are there inside GitHub Actions, and I'm using that predefined action."

Step type	Looks like	Does
`run:`	`run: ./build.sh`	Runs **your** shell command
`uses:`	`uses: actions/checkout@v6`	Runs a **ready-made action** someone published (`owner/repo@version`)
`with:`	under a `uses:`	That action's **inputs**

12.3 The artifact job

She told everyone to copy the build job from `08-artifacts` **and** its `build.sh` (*"I did not copy it — let me copy it"*):

```yaml
  artifact:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v6          # pull the repo onto the runner
      - name: Run build
        run: |
          chmod +x build.sh                # make it executable
          ./build.sh
      - name: Show build files
        run: |
          echo "Build output:"
          ls -la build                     # "not necessary, I just put it"
      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: session16-build            # → the zip's name
          path: build/                     # → what goes in it ("like a mount path")
```

This `build.sh` (the `08-artifacts` version, different from §5's) creates three files:

```
#!/bin/bash
set -e
echo "Starting build..."
rm -rf build
mkdir -p build
echo "Application version: 1.0.0" > build/version.txt
echo "Build status: SUCCESS" > build/build-info.txt
cat > build/app.txt <<EOF
Session 16 Calculator Application
This file was generated during the CI build.
EOF
echo "Build completed successfully."
```

WHY IT WORKS

Why `actions/checkout` is the first step. A fresh runner is an **empty** Ubuntu VM. Your repo is **not** on it. The `hello` job never noticed because `echo`, `date` and `uname` don't need your files. The moment a step says `./build.sh`, the file has to be there, and `actions/checkout@v6` is what clones your repo into the runner's working directory. Forget it and you get `chmod: cannot access 'build.sh': No such file or directory`.

WATCH IT HAPPEN

Watch it happen — from push to a zip in your Downloads folder.

```
Frame 1  git push  (commit "Artifact")
Frame 2  Fresh runner VM — empty
Frame 3  actions/checkout@v6        → repo files now on the VM
Frame 4  chmod +x build.sh; ./build.sh
         build/ ├── version.txt
                ├── build-info.txt
                └── app.txt
Frame 5  ls -la build               → prints the three files in the log
Frame 6  actions/upload-artifact@v4 → zips build/ as "session16-build"
         and stores it on GitHub
Frame 7  Job ends, VM destroyed — but the zip survives
Frame 8  Run summary: "Artifacts: 1" (was "none" in §9)
Frame 9  Click session16-build → download → unzip → 3 files ✅
```

SAID IN CLASS

"Can you see artifact is not null now? There is a one… Session 16 hyphen build — the same name we gave as an artifact… now you can download it… it's unzipped… See, these three files. app.txt, version.txt and build-info. This is the file that we created when we run the build.sh file."

NOTE

Your version matches hers exactly (`hey-cicd`, 16:18): same job, same `build.sh`, same artifact name `session16-build`. ✅

DO THIS YOURSELF

Lab 6 — artifacts.

1. Add the §12.3 `build.sh` to your repo root and the `artifact` job to your workflow.
2. Push. Open the run and confirm **Artifacts: 1** at the bottom of the summary.
3. Download `session16-build`, unzip it, and `cat` all three files.
4. **Delete the `Checkout code` step**, push, and read the failure. Then put it back.
5. Change `path: build/` to `path: build/version.txt`. Run and download. Now only one file is in the zip.

---

13 🧪 The Homework Pipeline — `demo/` and `needs`

13.1 What she assigned

SAID IN CLASS

"Create one GitHub repo. In that repo, clone this demo folder, and then you need to run your CI/CD pipeline… you will see two type of jobs. One is a test application and one is a build… Previously whenever you are running the job there is no joint. See, you are showing joint here… Because we added one parameter — the test application will finish, then and then build application will work. … Your task is to run this complete project, analyze this project and create a README or something like research.md file where you need to mention what you learned."

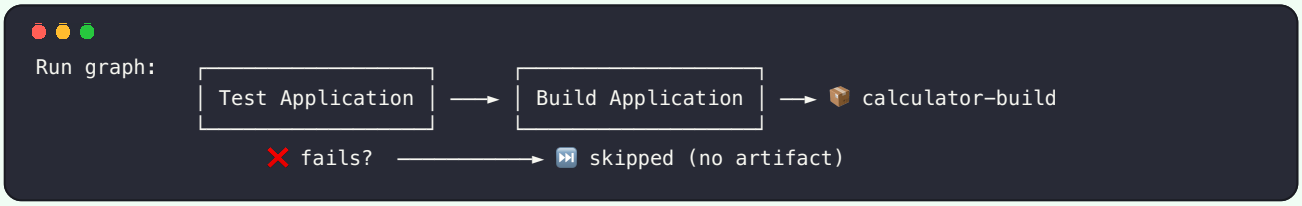
13.2 The file (`demo/.github/workflows/ci.yml`, identical to hers)

```yaml
name: Python CI Pipeline
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:
jobs:
  test:
    name: Test Application
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v6
      - name: Setup Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.12"
      - name: Display Python version
        run: python --version
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Run tests
        run: |
          pytest -v
  build:
    name: Build Application
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v6
      - name: Setup Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.12"
      - name: Build application
        run: |
          chmod +x build.sh
          ./build.sh
      - name: Show build output
        run: |
          ls -la build
          cat build/build-info.txt
      - name: Upload build artifact
        uses: actions/upload-artifact@v4
        with:
          name: calculator-build
          path: build/
```

LOCK THIS IN

Homework answer: what this pipeline teaches (try writing your own `research.md` first, then compare):

Thing to notice	What it means
Three triggers under one `on:`	Runs on push to `main`, on any **pull request** into `main`, **and** has a manual Run button
Two jobs: `test` and `build`	Each gets its own fresh VM, so **each** checks out the code and sets up Python again
`name: Test Application`	A job's display name can differ from its ID (`test`). `needs:` uses the **ID**
`needs: test`	This is the "*joint*": `build` waits for `test`. **If tests fail, build is skipped**, so broken code never gets packaged
`actions/setup-python@v7` + `python-version: "3.12"`	Pins the Python version instead of trusting whatever the runner has
`python -m pip install --upgrade pip`	Fresh pip before installing `requirements.txt`
`pytest -v`	Runs the 5 tests. Any failure → non-zero exit → red job
`cat build/build-info.txt`	Prints the build date (in **UTC**, since it's the runner's clock)
`upload-artifact` → `calculator-build`	The packaged app, downloadable as a zip



The one-line takeaway: CI isn't about getting a green tick. It's about **stopping** broken code before it moves forward.

TRY IT YOURSELF

Try it yourself: the failure demo from her repo's README (§39 "Demonstrate Failure"; not done live in class). Change `add` to `return a + b + 1`, run `pytest` locally (→ `FAILED tests/test_calculator.py::test_add`), then push. In Actions, **Test Application** fails at *Run tests* and **Build Application is skipped**. Fix it and push again → both green. Her own line for this: "*CI is not here to make developers happy. CI is here to catch broken code before it moves forward.*"

TRY IT YOURSELF

Extension from her README (§46): add `def power(a, b): return a ** b` and a `test_power` asserting `power(2, 3) == 8`. `pytest` should show **6 passed**. Push, check both jobs go green, and download `calculator-build`.

GOOD TO KNOW

> **Small observation on the file.** The `build` job runs `setup-python`, but `build.sh` is pure bash. It only copies a file and writes text, so that step isn't needed there. It's harmless (a few extra seconds) and future-proofs the job if the build ever needs Python.

13.3 Review of your `demo/` folder

What's there	Verdict
`app/`, `tests/`, `requirements.txt`, `build.sh`, `ci.yml`	✅ Byte-for-byte her `demo/`, so tests will pass
`build/` with `calculator.py` + `build-info.txt` (15:00:39 IST)	✅ Proof you ran `build.sh` locally
`.gitignore` lists `build/`	✅ Good. Build output belongs in **artifacts**, not in git. The pipeline regenerates it every run
Virtual env at `path/to/venv/`	⚠️ Her README's command was `python3 -m venv path/to/venv`, and `path/to/` is a **placeholder** you copied literally. It still works, and luckily the `venv/` line in `.gitignore` matches it at any depth, so it won't be committed. Next time use `python3 -m venv .venv`
That venv is **Python 3.9.6** (built from the repo's `.venv`)	💡 Fine locally. CI uses 3.12 via `setup-python`, so the two can differ. That's normal
`README.md` = her full 2,140-line teaching guide + `# hey-cicd` on the last line	📌 You merged her README with GitHub's auto-generated one. Worth trimming to your own notes or `research.md`
Git: one commit (`first commit`, 15:12, README only); everything else **untracked**. Remote: `hey-cicd`	⚠️ **Homework not pushed yet.** Push it to a **fresh** repo (not `hey-cicd`, which already holds the in-class pipeline), then watch the two jobs run
`CD/` folder	Empty. Ready for Class 18

14 Cheat-Sheet

Workflow anatomy

Key	Level	Meaning
`name:`	top	Workflow name in the Actions tab
`on:`	top	Trigger(s). **Only one** `on:`, but many triggers inside it
`jobs:`	top	Map of job ID → job
`<job-id>:`	jobs	e.g. `hello`, `test`, `build`. Referenced by `needs:`
`name:`	job	Display name (can differ from ID)
`runs-on:`	job	Runner: `ubuntu-latest`, `windows-latest`, `macos-latest`, `self-hosted`
`needs:`	job	Wait for another job; skipped if that job fails
`steps:`	job	Ordered list of steps
`- name:`	step	Label in the log
`run:`	step	Shell command (`run: |` for multi-line)
`uses:`	step	A published action, `owner/repo@version`
`with:`	step	Inputs to a `uses:` action
`env:`	step/job	Environment variables
`${{ secrets.NAME }}`	anywhere	Read a repository secret

Triggers

Trigger	Fires when
`workflow_dispatch`	You click **Run workflow** (manual)
`push: branches: [main, 'feature/*']`	A push to a matching branch
`pull_request: branches: [main]`	A PR into `main` is opened/updated
`schedule: - cron: '0 2 * * *'`	On a timer *(in her repo's `04-workflows`, not taught)*

Actions used today

Action	Does
`actions/checkout@v6`	Clones your repo onto the runner
`actions/setup-python@v7` (`with: python-version: "3.12"`)	Installs a specific Python
`actions/upload-artifact@v4` (`with: name, path`)	Saves files as a downloadable zip

Commands

Command	Use
`python3 -m venv .venv` · `source .venv/bin/activate`	Create / enter a virtual env
`pip install -r requirements.txt`	Install dependencies
`python3 app/calculator.py`	Run the app (`q` to quit)
`pytest` · `pytest -v`	Run tests · verbose
`chmod +x build.sh && ./build.sh`	Make executable and run the build
`git add . && git commit -m "…" && git push`	Trigger a push-based run
`echo` · `date` · `uname -a` · `ls -la`	The demo step commands
`[ -n "$X" ]` / `[ -z "$X" ]`	Bash: non-empty / empty test
`exit 1`	Fail the step on purpose
`gh workflow run <file>` · `gh run list` · `gh run view <id> --log`	Same things from the terminal *(in her repo README)*

GitHub UI paths

To…	Go to
Run manually	**Actions** → workflow name → **Run workflow** → branch
Add a secret	**Settings** → **Secrets and variables** → **Actions** → **New repository secret**
Re-run only failures	Run page → **Re-run jobs** ▾ → **Re-run failed jobs**
Download an artifact	Run page summary → **Artifacts** → click the name
See an auto-run	Repo home → 🟡/✅/❌ dot beside the latest commit

Concepts

Term	One line
CI	Build + test on every change
Continuous Delivery	Always releasable; **manual approval** before prod
Continuous Deployment	Every green build goes to prod automatically
Workflow	One YAML file in `.github/workflows/`
Job	A group of steps on **one** runner; parallel by default
Step	One command (`run`) or action (`uses`)
Runner	The machine that runs a job: GitHub-hosted VM or your own (self-hosted)
Secret	Encrypted, write-only value; masked as `***` in logs
Artifact	Files saved from a run, downloadable as a zip
Set up job / Complete job	Auto-added first/last steps (init + cleanup / "post job")

Quiz answers (§2)

Q	A
Course so far	Linux → Shell → Git/GitHub → Networking → Docker → Kubernetes → Helm
Docker objects	Image · Container · Network · Volume
K8s objects	Pod · ReplicaSet · Deployment · StatefulSet · DaemonSet · Service · ConfigMap · Secret · Ingress · PV · PVC · HPA
Service types (5)	ClusterIP · NodePort · LoadBalancer · ExternalName · Headless
Deployment strategies	Rolling update · Recreate · Blue-Green · Canary

15 📝 Homework

- [ ] **1. Run the demo pipeline.** Create a **new** GitHub repo, copy in the `demo/` folder (yours is ready, just untracked), push, and watch **Test Application** → **Build Application** run.
- [ ] **2. Analyse it.** How many jobs? Why is there a connecting line between them (`needs: test`)? What triggers it? What's in the artifact?
- [ ] **3. Write** `research.md` (or a README) in that repo: "*what you learned from this repo.*" The ✅ table in §13.2 is a model answer. Write yours first.
- [ ] *(Optional, from her README)* Break `add()` and watch Build get **skipped**; then add `power()` + `test_power` → **6 passed**.
- [ ] *(Fix-ups in your `hey-cicd`)* `branch:` → `branches:` , and `# needs: build` → `needs: build1` .

16 🗒️ Notes on the Transcript Itself

The instructor. The header credits **"Ritesh prajapati"**. That's the developer of the *Scaler++ Chrome extension* that downloaded the file, **not** the instructor. This class was taught by **Nensi Ravaliya** (she types her demo secret as `Nancy_PWD`, matching the "*Nancy*" the ASR always hears).

The title is correct. *CI/CD & GitHub Actions* is what was taught. Only the CD *deployment* part was theory-only. In her own numbering it's **session 16**, which is why the artifact is `session16-build` and the build file says `Session 16 Calculator`.

Transcription quality. `groq, whisper-large-v3-turbo`, so clean wherever she speaks English. The damaged stretches:

- **The opening quiz** is the messiest part: people answering over each other, her counting down at Team D, and a run of meaningless filler ("*Challenge 40… Travel with 1 second. Force… Seekers, go*") that is unrecoverable. The questions and the answers that were clearly audible are in §2.1.
- **Three Hindi stretches** were folded in: the projector/seating/attendance warning (§1), the mic-swapping aside (§1, mostly noise), and the Hinglish "*git push ho gaya? … hands up… go to your code section… Hello GitHub Actions dikh raha hai? click karo*" during the first run (§8.1).
- **The lab-time gaps** (~20 minutes across three stretches while she walked between desks) were transcribed as repeated "*Thank you*", "*foreign*", Spanish "*Gracias*", Portuguese "*E aí… Obrigado*", and one Spanish fragment ("*¿Cuántos años se ha empezado? … manifest one point one zero one five seven*"). All of it was skipped as noise. Nothing technical was lost: your git commits show these were the minutes you spent pushing.
- **The meeting interruption** before artifacts ("*Pauan sir? Shithith sir… Mail is going to go… I have a meeting with the director… with Dean*"). Kept briefly in §1; the names are unverified.
- **The final lines** ("*Aaj ka session ka hit nahi tha*" / "*Shallow, ablate the attendance. Khush hojaan*") are roughly "*Chalo*, attendance… be happy". The exact wording is unclear.

Cross-referenced against files, not just audio: every YAML block in §7–§13 matches her repo (`03-github-actions`, `07-secrets`, `08-artifacts`, `demo/`) **and** your `hey-cicd` history. The timings in the header and §1 come from your commit timestamps. The `build-info.txt` date is from your `demo/build/`.

Garbled-term corrections:

Transcript says	Actually
"*CACD*" / "*CICT*" / "*EACD*" / "*CACD is a whole divorce*"	**CI/CD**
"*Socle CA*"	**CircleCI**
"*Code build and code deploy*"	**AWS CodeBuild & CodeDeploy**
"*peep install-requirement.txt*" / "*requirement.php*"	`pip install -r requirements.txt`
"*Add, subscribe and then multiply*"	add, **subtract**, multiply
"*PyTest, type on V. V for jobless mode*"	`pytest -v`, **verbose** mode
"*build on SS / build.sh5 / build on a search file*"	`build.sh`
"*dot bit of folder*" / "*.Github workflows*"	`.github/workflows/`
"*method of action*" / "*a bit of action*" / "*bit of actions*"	**GitHub Actions**
"*unim-name … username*"	`uname -a` (prints the **OS/kernel**, not a username)
"*hey cacd*" / "*heycii cd.yaml*"	**hey-cicd** / `hey-cicd.yaml`
"*Shashi number 16*" / "*Shashan*"	**session** 16
"*Nancy underscore PWD*"	the demo secret value `Nancy_PWD`
"*secretly artifact*"	the **artifact** job ("*not secret demo — artifact*")
"*depends on parameter*" / "*joints*"	`needs:` and the connecting line it draws
"*Sherlock with add get commit*"	"*Chalo*", **git add, git commit**…
"*Diplomen*t"	**Deployment**
"*kittest objects*"	**Kubernetes** objects
"*Minicube*" / "*cubectl*"	**Minikube** / **kubectl**
"*Demon said*" / "*demon set*"	**DaemonSet**
"*Replica said*" / "*Stateful said*"	**ReplicaSet** / **StatefulSet**
"*kind map*"	**ConfigMap**
"*adlis*"	**headless**
"*blueprint*"	**blue-green**
"*anti-gravity*"	**Antigravity** (the IDE)
"*Oc5*" (lab-time fragment)	unrecoverable